

EV Charger Edge Node Firmware

Problem Statement

Develop firmware for an EV charger edge node that monitors an active charging session. The firmware should simulate voltage and current measurements, filter noisy sensor data, calculate power, detect fault conditions, and communicate system status using CAN and UART interfaces.

Objectives

The main objective of this project is to develop firmware for an EV charger edge node that monitors an active charging session in real time.

- Simulate output voltage (200V–500V) and output current (0A–100A).
- Add noise to the simulated signals and apply filtering to obtain stable readings.
- Calculate instantaneous power using $P = V \times I$.
- Detect overvoltage, overcurrent, and overpower conditions.
- Transmit voltage, current, power, and fault information through CAN messages at fixed intervals.
- Display fault and error messages through UART.
- Handle sensor and communication failures without stopping system operation.

A1. System Overview

Platform and Language

- Platform: ESP32
- Language: C
- Simulation Environment: Wokwi

Reason:

- ESP32 supports UART communication and CAN (TWAI) communication.
- It is easy to simulate using Wokwi.
- C is commonly used for embedded firmware development and provides efficient hardware control.

Major Components of the Firmware

1. Sensor simulation module(noise generation)
2. EMA filtering module
3. Fault detection module
4. CAN communication module
5. UART logging module
6. Error handling module

Assumptions

- Voltage range is 200V to 500V.
- Current range is 0A to 100A.
- Signals are simulated in software.
- Data is sampled every 100 ms.
- Loopback mode in CAN may be used for testing if supported by the simulation environment.
- UART is used for system monitoring, fault reporting, and debugging through the Serial Monitor.
- The system operates continuously during an active charging session.
- Sampling Timing:
Sampling Interval = 100 ms
Sampling Frequency = 10 Hz

A2. Data and Signal Design

Signal Representation

The following parameters are transmitted through CAN:

- Voltage
- Current
- Power
- Fault status

All values are represented using unsigned integer data types.

Filter Selection

- Exponential Moving Average (EMA) filter is used.
- EMA reduces noise while maintaining a good response to signal changes.
- It requires very little memory and processing overhead.

Filtered Value = $\alpha \times \text{Current Reading} + (1 - \alpha) \times \text{Previous Filtered Value}$

EMA Configuration:

Filter Coefficient: Alpha = 20%

- The filter gives 20% weight to the current reading and 80% weight to the previous filtered value.

- This provides good noise reduction while maintaining reasonable responsiveness to signal changes.

Fault Thresholds

<u>Fault Type</u>	<u>Threshold</u>
Overvoltage	> 480V
Overcurrent	> 90A
Overpower	> 22000W

(A fault is triggered when the voltage exceeds 480V, current exceeds 90A, and power exceeds 22KW)

Reason for Selection:

- The threshold values were chosen based on the limits suggested in the assignment.
- These values represent safe operating limits for monitoring the charging session.
- When any threshold is exceeded, a fault message is reported through UART and the corresponding fault flag is set in the CAN frame.

A3. CAN Frame Design

CAN Identifier

- Standard CAN Frame (11-bit Identifier)
- CAN ID: 0x100

A standard CAN identifier is sufficient because only a small number of messages are transmitted in this application.

The CAN payload is limited to 8 bytes in Classical CAN. Therefore, voltage, current, power, and fault information are packed efficiently into an 8-byte CAN frame.

CAN Payload Layout:

<u>Byte</u>	<u>Description</u>
0-1	Voltage
2-3	Current
4-5	Power
6	Fault Flags
7	Reserved(value =0)

- CAN payload generation and frame packing are implemented in software.
- Actual CAN loopback verification depends on simulator support.

Bit Width and Scaling

Voltage: 16-bit unsigned integer

Current: 16-bit unsigned integer

Power: 16-bit unsigned integer

Fault Flags: 8-bit field

- *Voltage:*

- Range: 200V – 500V
- Maximum value is 500.
- One byte can store only 0–255.
- Therefore, a 16-bit (2-byte) unsigned integer is used.

- *Current:*

- Range: 0A – 100A.
- Although 1 byte is sufficient, a 16-bit (2-byte) unsigned integer is used for consistency in the CAN frame.

- *Power:*

- Maximum power = $500V \times 100A = 50000W$.
- One byte cannot store this value.
- Therefore, a 16-bit (2-byte) unsigned integer is used.

- *Fault Flags:*

- Stored using 1 byte (8 bits).
- Individual bits indicate Overvoltage, Overcurrent, and Overpower faults.

Fault flag bits:

Bit 0 – Overvoltage

Bit 1 – Overcurrent

Bit 2 – Overpower

Scaling

- Voltage: 1:1 scaling
- Current: 1:1 scaling

- Power: 1:1 scaling

The actual values are transmitted directly without any additional scaling factor.